```
Start coding or generate with AI.
```

Name: Aman Shaikh

Roll No: F-02

Problem Statement:

Implement feature scaling techniques (Standardization and Min-Max Scaling) and observe their effect on model performance.

Aim

To study and implement feature scaling techniques using StandardScaler and MinMaxScaler and compare the performance of a K-Nearest Neighbors (KNN) classifier before and after scaling.

```
# Train KNN on Min-Max scaled data
knn_minmax = KNeighborsClassifier(n_neighbors=5)
knn_minmax.fit(X_train_scaled_minmax, y_train)

# Make predictions on Min-Max scaled test data
y_pred_minmax = knn_minmax.predict(X_test_scaled_minmax)

# Evaluate performance on Min-Max scaled data
accuracy_minmax = accuracy_score(y_test, y_pred_minmax)
print(f"Accuracy on Min-Max scaled data: {accuracy_minmax:.4f}")
```

```
Accuracy on Min-Max scaled data: 0.9444
```

```
# Train KNN on standardized data
knn_standard = KNeighborsClassifier(n_neighbors=5)
knn_standard.fit(X_train_scaled_standard, y_train)

# Make predictions on standardized test data
y_pred_standard = knn_standard.predict(X_test_scaled_standard)

# Evaluate performance on standardized data
accuracy_standard = accuracy_score(y_test, y_pred_standard)
print(f"Accuracy on standardized data: {accuracy_standard:.4f}")
```

```
Accuracy on standardized data: 0.9630
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Initialize KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)

# Train KNN on original (unscaled) data
knn.fit(X_train, y_train)
```

```
# Make predictions on original test data
y_pred_original = knn.predict(X_test)

# Evaluate performance on original data
accuracy_original = accuracy_score(y_test, y_pred_original)
print(f"Accuracy on original data: {accuracy_original:.4f}")
```

```
Accuracy on original data: 0.7407
```

```
from sklearn.preprocessing import MinMaxScaler

# Initialize MinMaxScaler
minmax_scaler = MinMaxScaler()

# Apply MinMaxScaler to the training data
X_train_scaled_minmax = minmax_scaler.fit_transform(X_train)

# Apply MinMaxScaler to the test data
X_test_scaled_minmax = minmax_scaler.transform(X_test)

# Display the first 5 rows of the Min-Max scaled training data
print("Min-Max Scaled X_train (first 5 rows):\n", X_train_scaled_minmax[:5])
print("\nMin of X_train_scaled_minmax:\n", X_train_scaled_minmax.min(axis=0))
print("\nMax of X_train_scaled_minmax:\n", X_train_scaled_minmax.max(axis=0))
```

```
Min-Max Scaled X_train (first 5 rows):
 [[0.64736842 0.54989817 0.44385027 0.45876289 0.19565217 0.22068966
  0.02953586 0.8490566  0.14556962 0.43708609 0.26829268 0.2014652
  0.23798266]
 [0.38947368 0.17107943 0.3315508  0.51030928 0.16304348 0.42068966
  0.33333333 0.35849057 0.33544304 0.13245033 0.45528455 0.84249084
  0.31048069]
 [0.34210526 0.0203666  0.31550802 0.21649485 0.7173913  0.31724138
  0.3185654  0.41509434 0.74050633 0.18322296 0.47154472 0.38095238
  0.37194641]
 [0.59210526 0.15274949 0.79144385 0.25257732 0.43478261 0.55862069
  0.49367089 0.39622642 0.29746835 0.31567329 0.49593496 0.55311355
  0.47438928]
 [0.33157895 0.39511202 0.45989305 0.3814433  0.19565217 0.50689655
  0.40295359 0.22641509 0.49683544 0.04525386 0.54471545 0.74358974
  0.00945626]]

Min of X_train_scaled_minmax:
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

Max of X_train_scaled_minmax:
 [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

```
from sklearn.preprocessing import StandardScaler

# Initialize StandardScaler
scaler = StandardScaler()

# Apply StandardScaler to the training data
X_train_scaled_standard = scaler.fit_transform(X_train)

# Apply StandardScaler to the test data
X_test_scaled_standard = scaler.transform(X_test)

# Display the first 5 rows of the standardized training data
```

```
print("Standardized X_train (first 5 rows):\n", X_train_scaled_standard[:5])
print("\nMean of X_train_scaled_standard:\n", X_train_scaled_standard.mean(axis=0))
print("\nStandard Deviation of X_train_scaled_standard:\n", X_train_scaled_standard.st
```

```
Standardized X_train (first 5 rows):
 [[ 0.62844732  1.08120605 -0.65212742  0.         -0.8414766  -1.00335756
  -1.51706225  1.71144809 -1.23077056  0.33317435 -0.64137827 -1.07090115
  -0.51821917]
 [-0.54088231 -0.61299439 -1.42753372  0.28817977 -1.03748743 -0.11258512
  -0.08675106 -0.35047592 -0.19503558 -0.93349517  0.34652993  1.33076844
  -0.21506322]
 [-0.75565714 -1.28703113 -1.53830605 -1.35444491  2.29469668 -0.57332948
  -0.15628008 -0.11256161  2.01453238 -0.72238359  0.43243499 -0.39843366
   0.04196031]
 [ 0.37787668 -0.69497183  1.7479397  -1.15271907  0.59593615  0.50174071
   0.6681354  -0.19186638 -0.40218258 -0.1716577   0.56129258  0.24658617
   0.47033286]
 [-0.80338488  0.3889521  -0.54135509 -0.43226965 -0.8414766   0.27136852
   0.24102858 -0.9056093   0.68533915 -1.29605638  0.81900777  0.96022513
  -1.47381948]]

Mean of X_train_scaled_standard:
 [ 2.55440830e-15 -7.03906019e-16  2.92642255e-15 -2.46218816e-17
  1.93617523e-16  1.19528043e-15  1.77949053e-17 -9.02056208e-16
  6.80906944e-16  9.92037993e-16  2.35116586e-15  1.51491722e-15
  6.35692216e-17]

Standard Deviation of X_train_scaled_standard:
 [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

```
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (124, 13)
X_test shape: (54, 13)
y_train shape: (124,)
y_test shape: (54,)
```

```
import pandas as pd
from sklearn.datasets import load_wine

# Load the Wine dataset
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target)

print("Features:\n", X.head())
print("\nTarget classes:\n", wine.target_names)
```

```
Features:
    alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80
```

```
3    14.37        1.95  2.50               16.8      113.0           3.85
4    13.24        2.59  2.87               21.0      118.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
0        3.06                  0.28             2.29             5.64  1.04
1        2.76                  0.26             1.28             4.38  1.05
2        3.24                  0.30             2.81             5.68  1.03
3        3.49                  0.24             2.18             7.80  0.86
4        2.69                  0.39             1.82             4.32  1.04

   od280/od315_of_diluted_wines  proline
0                          3.92   1065.0
1                          3.40   1050.0
2                          3.17   1185.0
3                          3.45   1480.0
4                          2.93    735.0

Target classes:
 ['class_0' 'class_1' 'class_2']
```